

Sistemas Multiagente

Juegos Alternados Implementación

Sergio Yovine

Universidad ORT Uruguay
yovine@ort.edu.uy

6 de mayo de 2026

Implementación: planificar y jugar

$P \leftarrow \text{Players}()$

$G \leftarrow \text{Game}()$

while $\neg G.\text{EsFinal}()$ **do**

$p \leftarrow G.\text{player}()$

 (* Planificar *)

$a \leftarrow p.\text{action}(G)$

 (* Jugar *)

$G.\text{play}(a)$

El juego G mantiene un *estado* s y el jugador p en s

Implementación: Minimax

La función $p.action$ para Minimax es:

```
(* Precondición:  $\neg G.EsFinal() \wedge d_{max} > 0$  *)
```

```
(* Calcular el valor y la mejor acción *)
```

```
 $a, v \leftarrow p.minimax(G, d_{max})$ 
```

```
(* Retornar la mejor acción *)
```

```
return  $a$ 
```

Implementación: Minimax

La función $p.\text{minimax}(G, d)$ es:

```
if  $G.\text{EsFinal}()$  then
    return None,  $G.\text{Utilidad}(p)$ 

if  $d = 0$  then
    return None,  $G.\text{Eval}(p)$ 

 $V \leftarrow []$ 
for  $a \in G.\text{actions}()$  do
     $G_a \leftarrow G.\text{clone}()$ 
     $G_a.\text{play}(a)$ 
     $\rightarrow, v_a \leftarrow p.\text{minimax}(G_a, d - 1)$ 
     $V \leftarrow V.\text{append}((a, v_a))$ 

if  $p = G.\text{player}()$  then
    return  $a, v_a$  such that  $v_a = \text{máx } V$  (*  $\pi_{\text{minimax}}$  *)
else
    return  $a, v_a$  such that  $v_a = \text{mín } V$  (*  $\pi_{\text{min}}$  *)
```